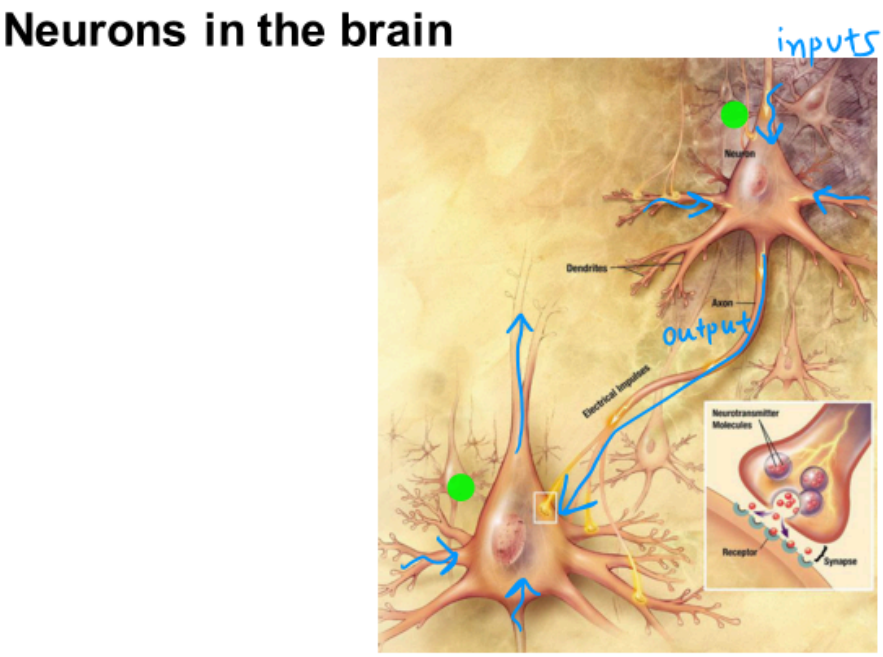
C2_Week1

▼ Neurons and the brain

Phần này giới thiệu sơ qua về các thuật toán cố bắt chước cách một chiếc não hoạt động, mở đầu vào các năm 80s tới 90s, nhưng sau đã bớt thịnh hình vào cuối các năm 90s ấy.

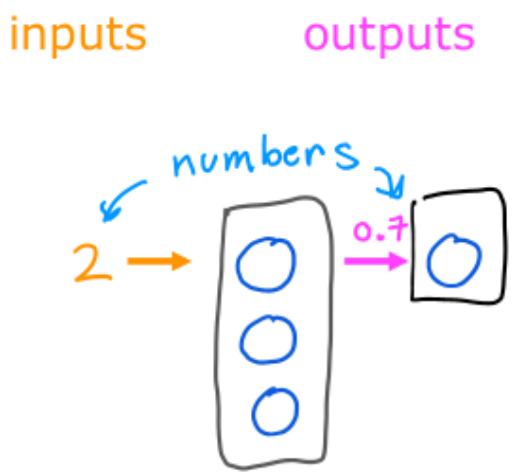
→ Vào khoảng 2005 đã quay lại, với các ứng dụng như nhận biết giọng nói, hình ảnh và ký tự.



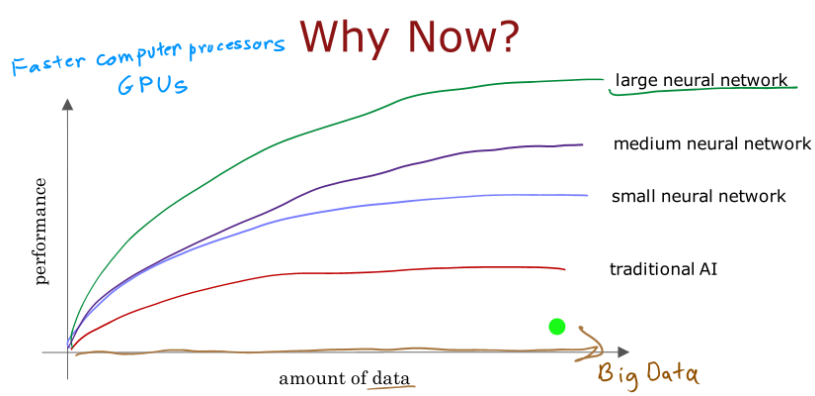
Hình ảnh Neurons ở trong não

Khi bắt chước một neuron trong chương trình máy tính, ta có một Simplified Mathematical model of a Neuron:

- Nhận vào input → tính toán → Output



- Lý do mình sử dụng mô hình này là với sự phát triển của Big Data, một hệ thống Large Neural Network sẽ đem tới Performance tốt nhất có thể với lượng Data ấy.



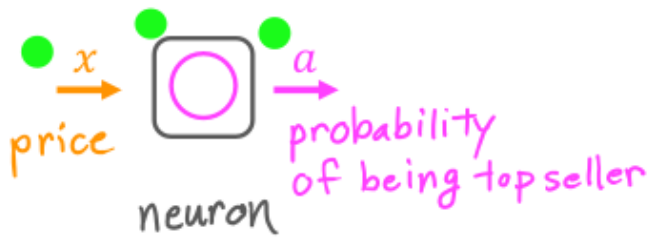
▼ Demand Prediction

Phần này viết về một bài toán dự đoán nhu cầu mua sắp của một sản phẩm nào đó.

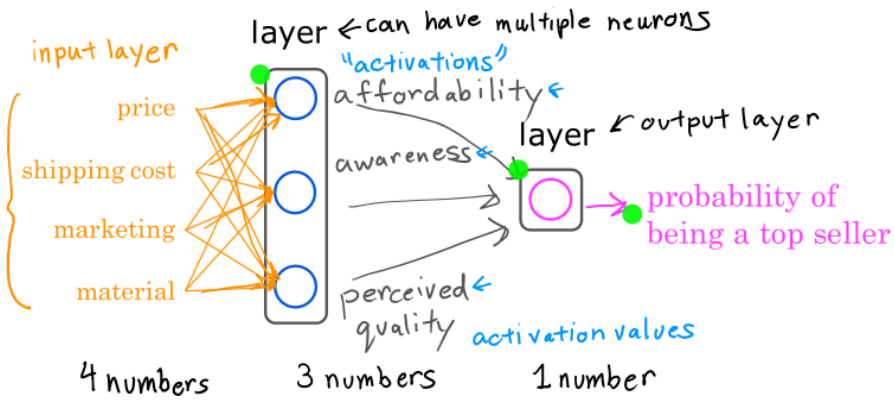
Ví dụ, để đưa ra được dự đoán ấy mình sẽ thử tính toán xem tỷ lệ của việc sản phẩm có phải là một Top-Seller hay không? Ta sẽ sử dụng một hàm Sigmoid, gọi là một Activation.

$$f(z) = \frac{1}{1 + e^{-z}}$$

- Việc sử dụng một Neuron sẽ được minh hoạ như này:



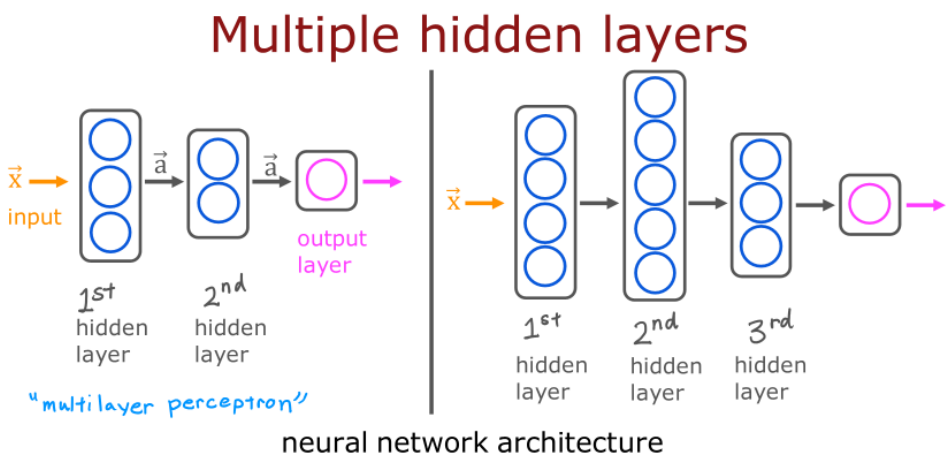
Với một hệ thống các Neurons nhiều và đầy đủ hơn, ta có thể sử dụng các yếu tố đầu vào như: giá cả, giá ship, marketing, chất liệu, đưa qua một "lớp" Neurons để tạo ra các thông số như: affordability, awareness, quality. Cuối cùng đưa vào một layer khác để tạo ra "tỷ lệ là một Top-Seller"



- Layer cuối cùng được gọi là Output Layer
- Một Layer có thể có nhiều Neurons (layer 1 có 3, layer 2 chỉ có 1)
- Các Layer ở giữa được gọi là "Hidden Layers"
- Activation là một hàm toán học (trong trường hợp này là Sigmoid) để đưa ra output của Neuron.

→ Mình có thể hiểu từ 4 output vừa rồi, việc mình đưa qua lớp Neuron đầu tiên để đưa ra các thông số mới ấy, tương đương với **Feature Engineering** ở Course 1.

Có thể có nhiều hidden layer như này:

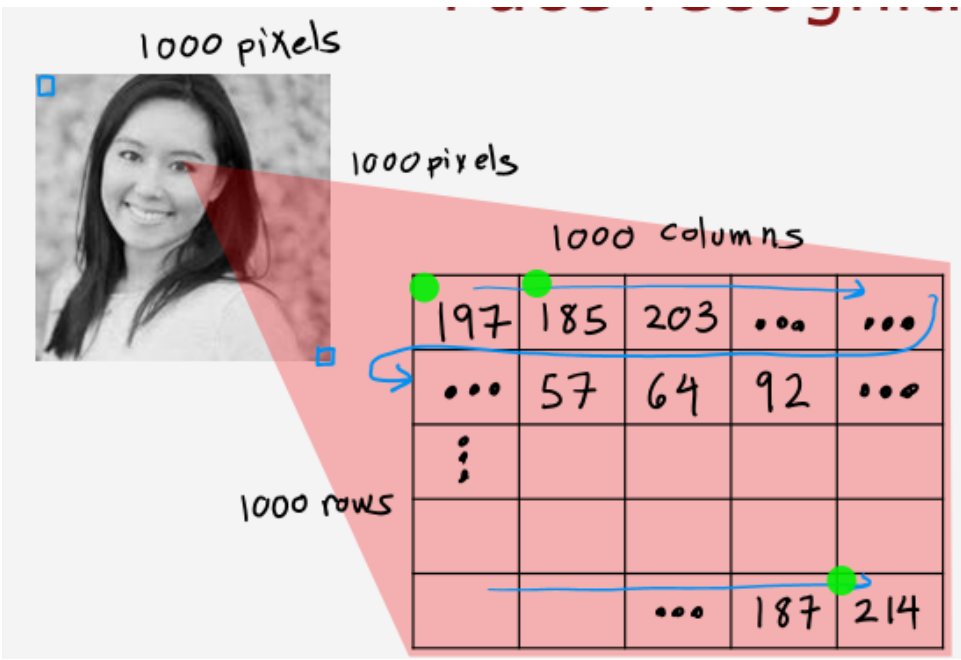


▼ Examples

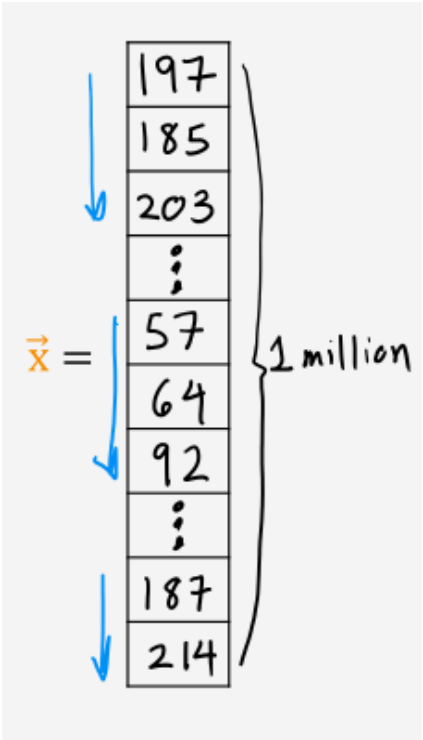
Trong phần này sẽ đưa ra 2 ứng dụng của một Neural Networks. Nhưng sẽ chỉ giải thích ví dụ đầu tiên về nhận diện khuôn mặt, ví dụ thứ 2 về nhận diện ô tô tương tự.

1. Ứng dụng trong nhận diện khuôn mặt:

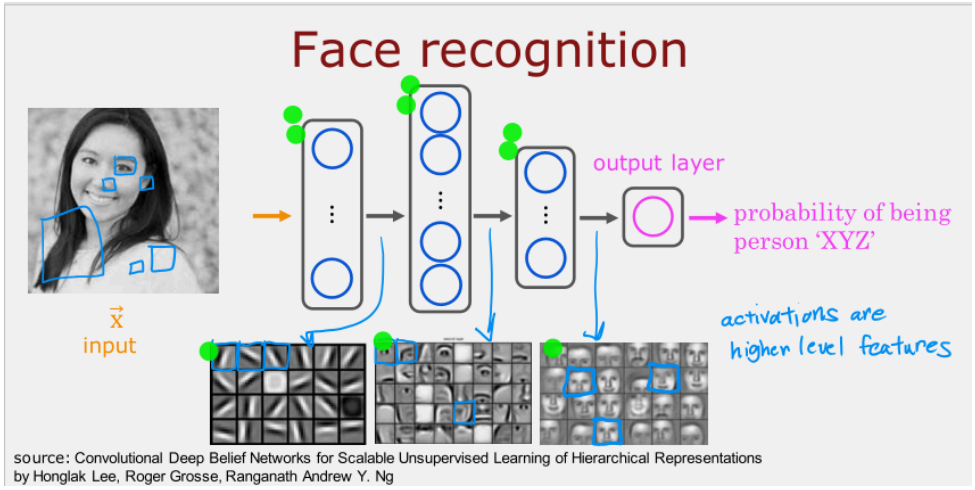
- Tưởng tượng mình có một bức ảnh, với kích thước 1000 × 1000 pixels, khi mình lấy bức ảnh làm đầu vào của Model, bức ảnh sẽ thành một chiếc bảng chứa mức độ sáng của từng pixel trong ảnh.



- Để viết dễ hơn mình sẽ đại diễn thông tin này bằng một vector có 1000×1000 = 1 triệu phần tử:

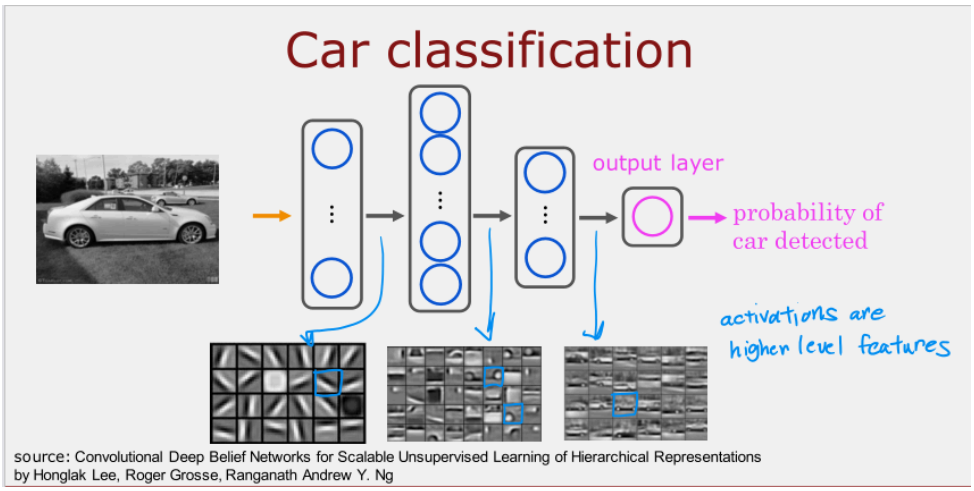


- Đây là slide miêu tả điều gì sẽ xảy ra khi mình input thông tin này vào một Neural network:

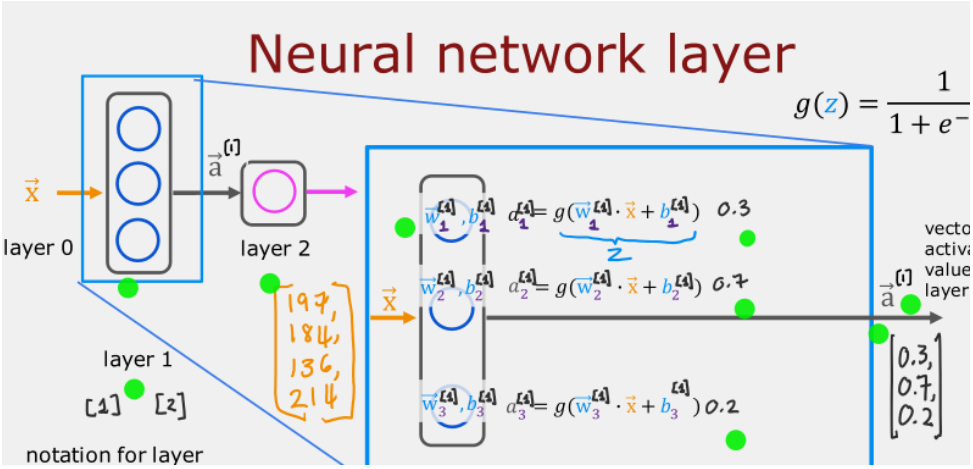


- Layer 1: các đường vẽ, hoặc cạnh/ nét, các vùng sáng hoặc tối gần với nhau.
- Layer 2: Ghép các đường nét/ cạnh ấy thành tổ hợp các hình phức tạp hơn như góc mũi, vành tài (kiểu các bộ phận nhỏ của mặt)
- Layer 3: Kết hợp các chi tiết ấy lại thành các bộ phận õ hơn như mắt, mũi, miệng...
- Output layer: tỷ lệ hoặc xác suất bức ảnh đấy là người "XYZ" (từ 0 - 1 vì hàm Sigmoid)

2. Nhận diện/ Phân loại xe ô tô (quy trình giống như trên)



▼ Neural Network Layer



Sau này sẽ sử dụng một số ký hiệu như sau:

- Một activation vector(một trọng số, bias, ...) ở một tầng x bất kỳ, sẽ có ký hiệu như này:

$$\vec{a}^{[x]}$$

- Do trong một số layer, sẽ có nhiều giá trị trọng số tương ứng với từng Neuron trong lớp ấy, ta sẽ viết:

$$w_1^{[1]}$$

→ Con số đại diện lớp nằm ở trên, số thứ tự nằm ở dưới (bên trên là weight số 1, ở layer 1)

- Công thức activation của các Neuron(1, 2, 3) được viết dưới dạng:

$$a_1^{[1]} = g\left(w_1^{[1]} \cdot \vec{x} + b_1^{[1]}\right)$$

> Tương tự với 2 neuron còn lại

- Sau khi chạy xong lớp thứ nhất mình sẽ có được một vector như:

$$\begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \end{bmatrix}$$

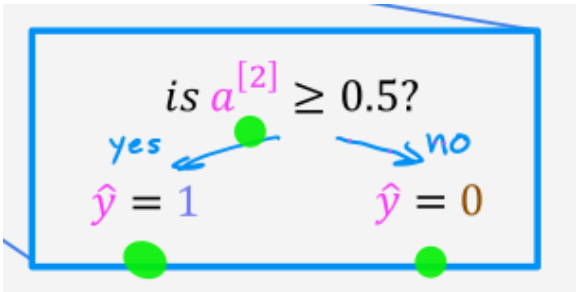
- Lưu ý rằng do đây là Layer 1 (layer ngay sau input) nên mình đang nhân tích vô hướng của trọng số với Input vector x.

→ Đối với công thức activation từ layer 2 trở đi phải thay vector x thành vector của lớp trước đó:

$$a_1^{[2]} = g\left(w_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}\right)$$

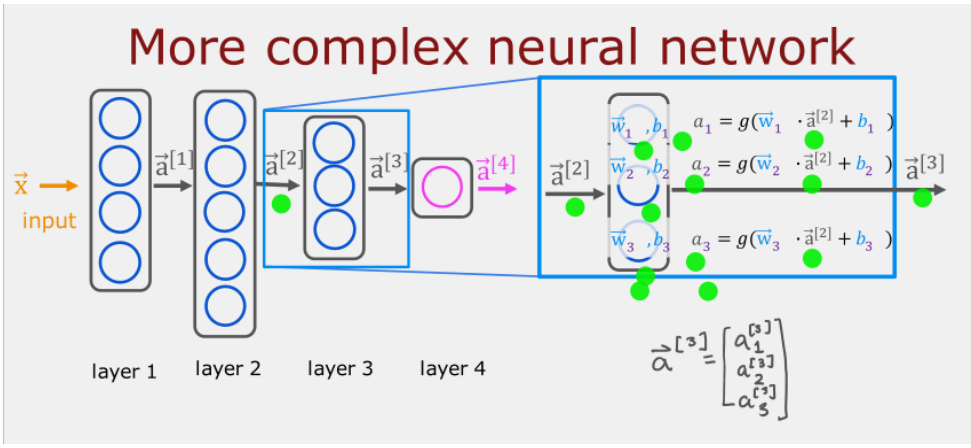
> Neuron số 1 của lớp số 2

- Sau khi mình chạy xong tới output layer thì mình có thể so sánh với một decision boundary (chẳng hạn 0.5) để đánh giá xem y=1 hay y=0.



▼ More Complex Neural Networks

Cùng với các cách ký hiệu như trên, một Neural Network phức tạp hơn sẽ nhìn như này:



Do ký hiệu khá nhiều nên mình có thể đưa ra một công thức tổng quá cho Output của một Neuron bất kỳ:

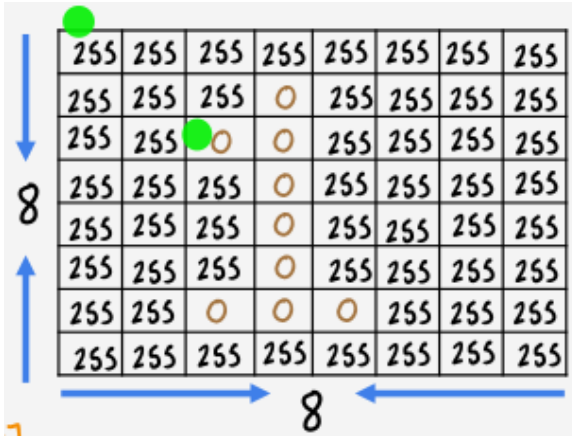
$$a_j^{[l]} = g\left(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]}\right)$$

Note:

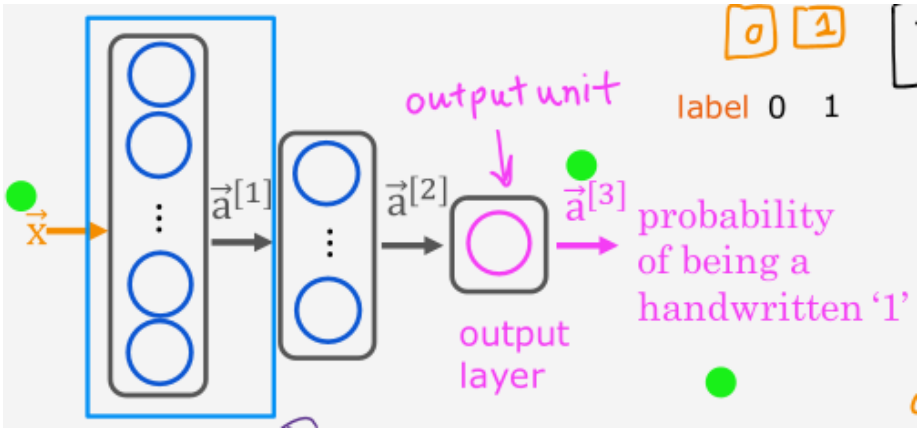
$a_j^{[l]}$: **giá trị activation (scalar)** của **neuron j ở layer l**
$\vec{w}_j^{[l]}$: **vector weight** của neuron đó
$\vec{a}^{[l-1]}$: **vector activation** của **layer trước (l−1)**
$b_j^{[l]}$: **bias (scalar)** của neuron đó

▼ Inference: Making Predictions (forward propagation)

Chẳng hạn với một bức tranh pixel 8×8 được trải thẳng thành vector có 64 phần tử để đưa vào Neural Network, mỗi phần tử nằm trong khoảng [0, 255] biểu thị mức độ sáng. Với 0 là màu đen, 255 là màu trắng.



- Và Neural Network có cấu trúc:



- layer 1: **25 units**
- layer 2: **15 units**
- layer 3 (output layer): **1 unit**

Activation vector của lớp số 1 sẽ có 25 phần tử, mỗi phần tử là output của hàm sigmoid: (tương tự với các lớp khác)

$$\vec{a}^{[1]} = \begin{bmatrix} g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]}) \\ \vdots \\ g(\vec{w}_{25}^{[1]} \cdot \vec{x} + b_{25}^{[1]}) \end{bmatrix}$$

Với sự lặp đi lặp lại qua từng lớp:

$$\vec{x} \rightarrow \vec{a}^{[1]} \rightarrow \vec{a}^{[2]} \rightarrow a^{[3]}$$

Ta gọi đây là forward propagation, quá trình đi từ trái sang phải của một mạng. Từ đó có thể quyết định output xem là 1 hoặc 0.

$$\hat{y} = \begin{cases} 1, & a^{[3]} \geq 0.5 \\ 0, & a^{[3]} < 0.5 \end{cases}$$